Procesos de Desarrollo de Software

Persistencia con JPA

Sesión Práctica

Contenidos

1. Motivación.
2. Ejemplo: Biblioteca.
3. Configuración.
4. Consultas con JPQL.
5. Repositorios.

1. Motivación

El problema de guardar datos en lenguajes orientados a objetos

1. Motivación – Definición

La **persistencia** es la capacidad de almacenar y recuperar datos de una aplicación de forma que sobrevivan a la ejecución del programa.

RETOS

Fricción objeto–relacional.
Las aplicaciones se diseñan con objetos, pero los datos se almacenan en tablas.

Rendimiento.
El acceso a la BBDD es mucho más costoso que trabajar en memoria.

Escalabilidad y volumen de datos.
Algunas aplicaciones manejan grandes volúmenes de datos (*Big Data*).

Gestión de concurrencia y consistencia.
Muchos usuarios pueden acceder y modificar los mismos datos.

Mantenibilidad del código.
Las consultas SQL pueden acoplar el código a la estructura de la base de datos.

1. Motivación – Definición

La **persistencia** es la capacidad de almacenar y recuperar datos de una aplicación de forma que sobrevivan a la ejecución del programa.

RETOS

Fricción objeto–relacional.
Las aplicaciones se diseñan con objetos, pero los datos se almacenan en tablas.

Rendimiento.
El acceso a la BBDD es mucho más costoso que trabajar en memoria.

Escalabilidad y volumen de datos.
Algunas aplicaciones manejan grandes volúmenes de datos (*Big Data*).

Gestión de concurrencia y consistencia.
Muchos usuarios pueden acceder y modificar los mismos datos.

Mantenibilidad del código.
Las consultas SQL pueden acoplar el código a la estructura de la base de datos.

1. Motivación – JPA

Java Persistence API (JPA) es la especificación Java acerca de cómo persistir objetos en BBDDs relacionales.

- Estandariza la estructura del mapeo objeto-relacional en Java
- Diferentes proveedores: Hibernate, EclipseLink, etc.

Terminología básica

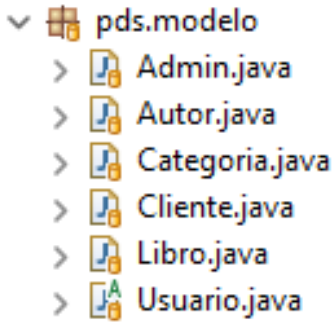
- Entity.
- EntityManager.
- Persistence Context.
- Criteria API.
- Persistence Unit.
- JPQL (*Java Persistence Query Language*).

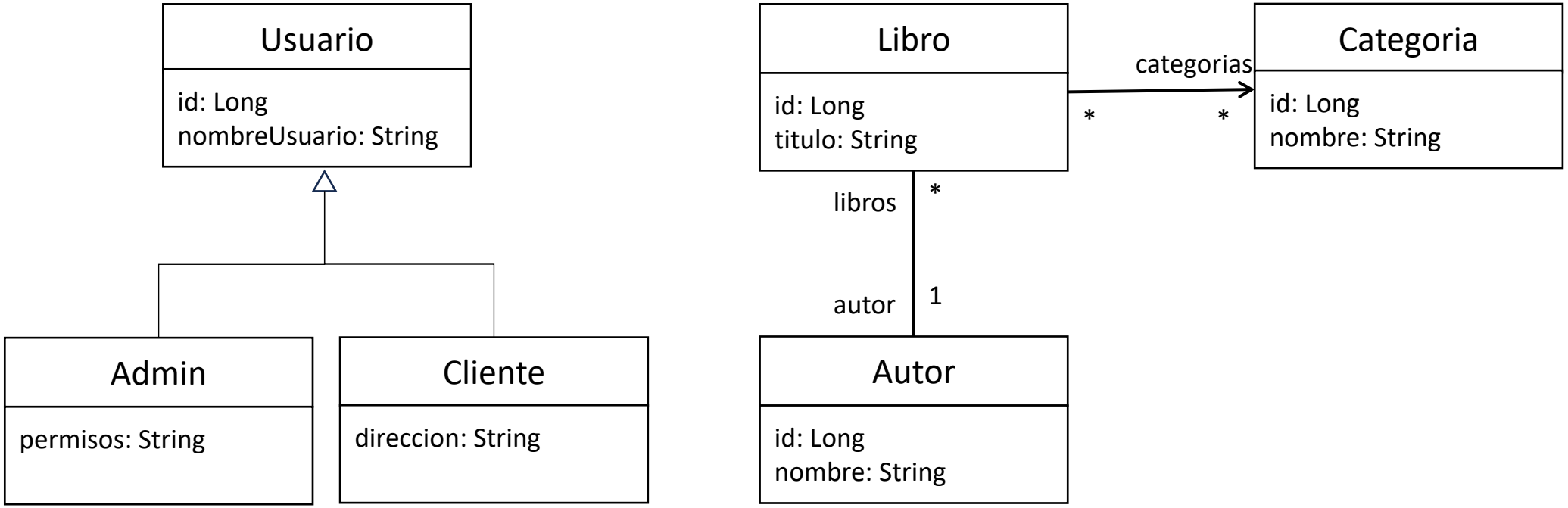
2. Ejemplo: Biblioteca

Ejemplo básico de uso de JPA

2. Ejemplo: Biblioteca

Modelo del dominio





2. Ejemplo: Biblioteca

Anotaciones básicas de una clase a persistir

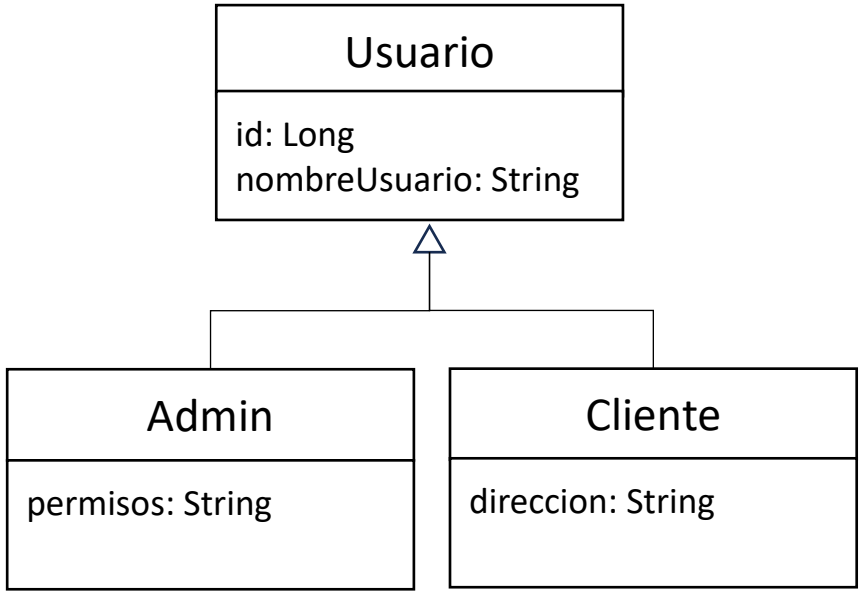
- Anotarla con @Entity para indicar que se quiere persistir
- (Opcional) Indicar el nombre de la tabla donde se va a persistir
- Indicar qué atributo será clave primaria
- Indicar estrategia para obtener clave primaria
  - IDENTITY: Usa columnas auto-incrementales
  - SEQUENCE: Usa secuencia de la bbdd
  - TABLE: Usa una tabla a modo de secuencia

```
@Entity
@Table(name = "authors")
public class Autor {
   @Id
   @GeneratedValue(
      strategy = GenerationType.IDENTITY)
   private Long id;
    . . .
}
```

2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

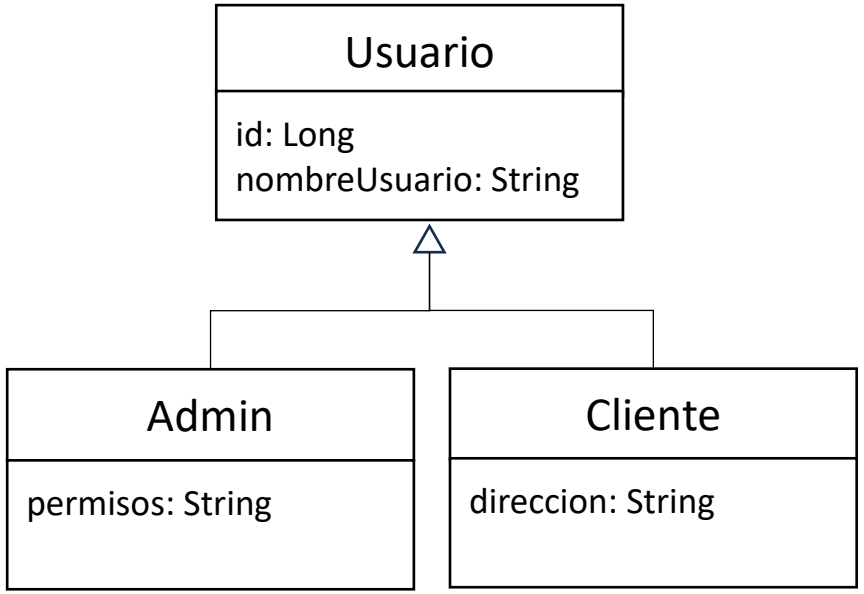
- SINGLE TABLE: Tabla única para toda la jerarquía
- JOINED: Tabla por clase
- TABLE_PER_CLASS: Tabla por clase concreta

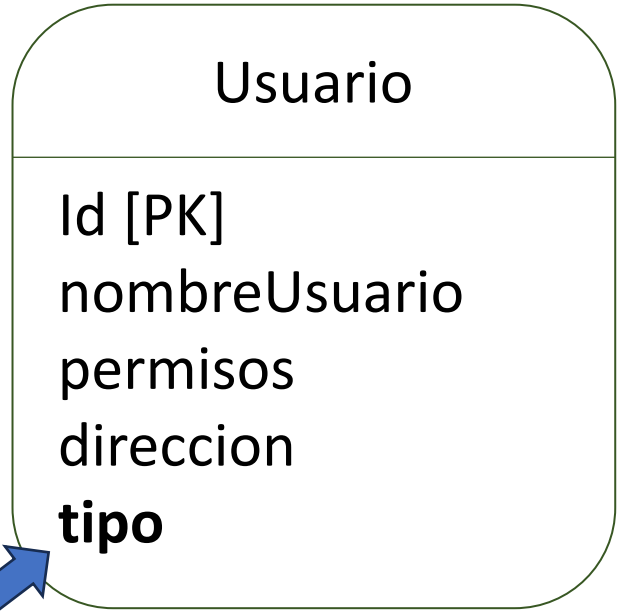


2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

- **SINGLE TABLE: Tabla única para toda la jerarquía**
- JOINED: Tabla por clase
- TABLE_PER_CLASS: Tabla por clase concreta



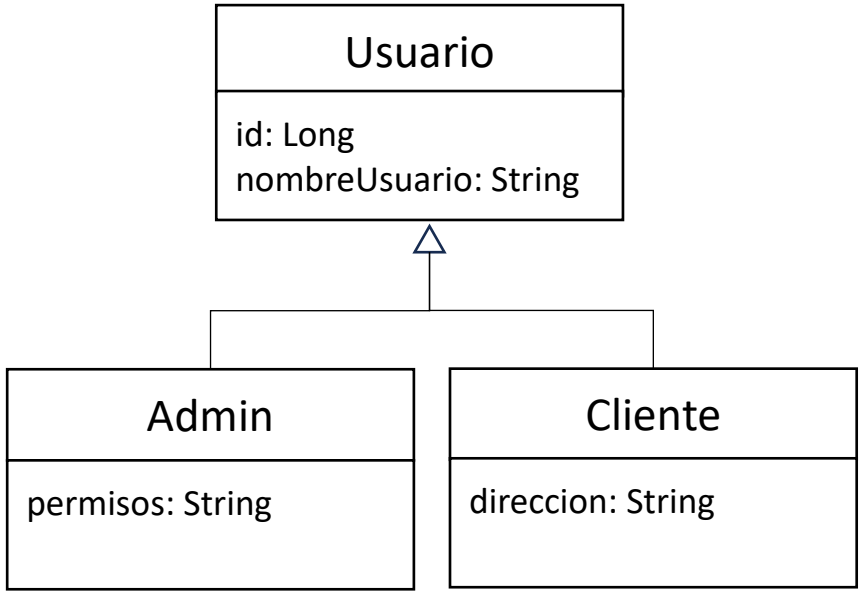


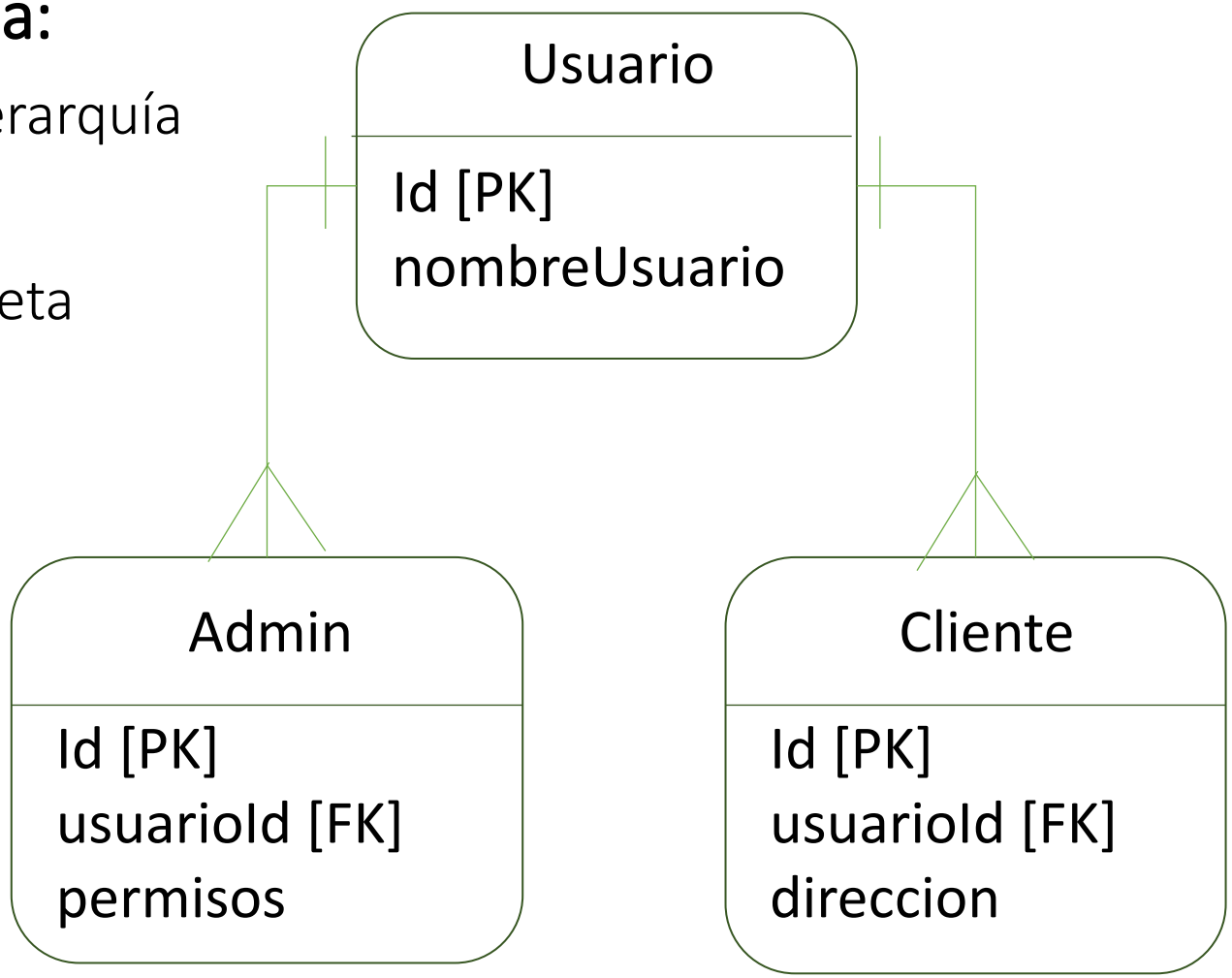
Necesario para recuperar la clase correcta de cada instancia

2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

- SINGLE TABLE: Tabla única para toda la jerarquía
- **JOINED: Tabla por clase**
- TABLE_PER_CLASS: Tabla por clase concreta

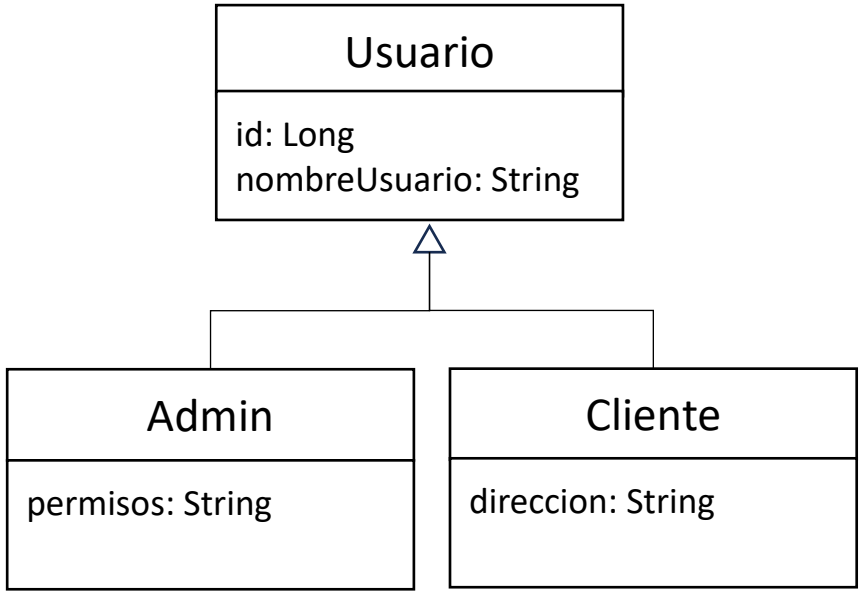


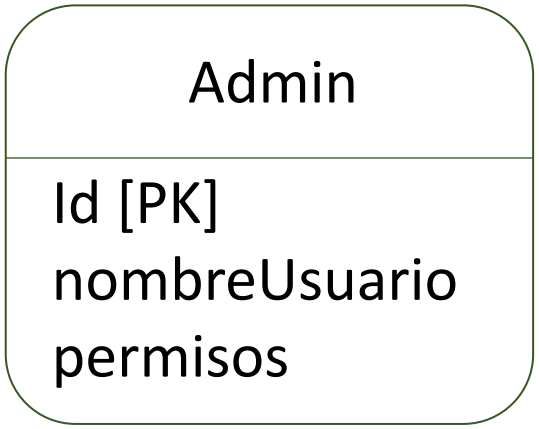


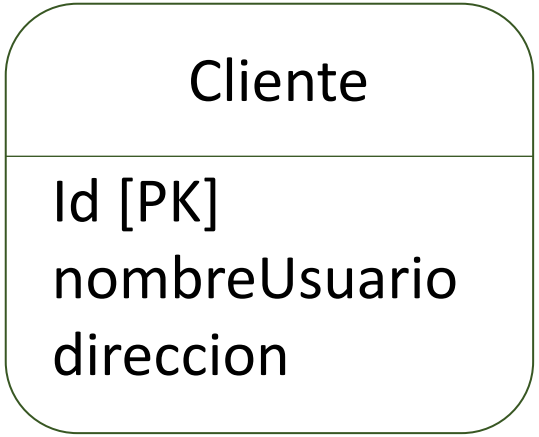
2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

- SINGLE TABLE: Tabla única para toda la jerarquía
- JOINED: Tabla por clase
- **TABLE_PER_CLASS: Tabla por clase concreta**



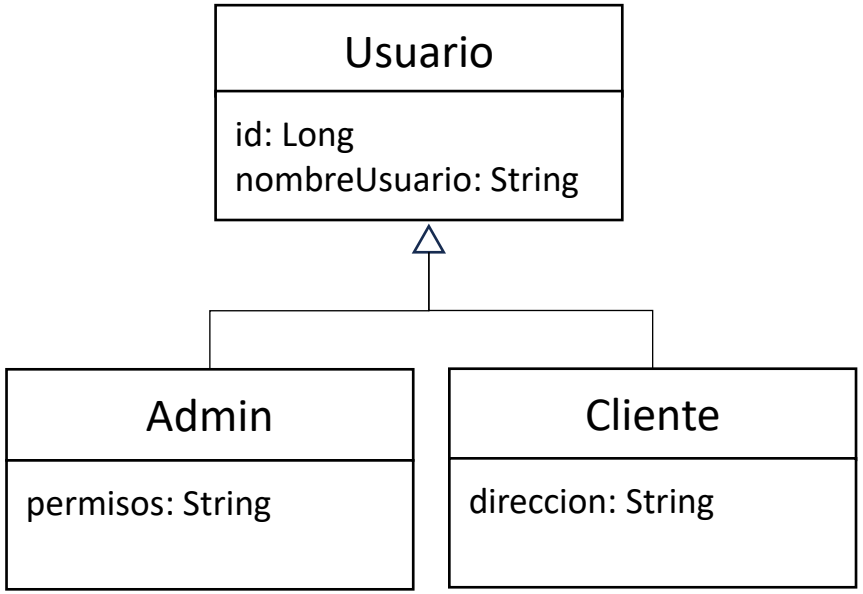




2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

- ¿Cuál hemos elegido para el ejemplo?



2. Ejemplo: Biblioteca

Estrategias para persistir una jerarquía:

- ¿Cuál hemos elegido para el ejemplo?

```
@Entity
@Table(name = "usuarios")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn( name = "tipo_usuario", discriminatorType = DiscriminatorType.STRING)
public abstract class Usuario {
. . .
}
```

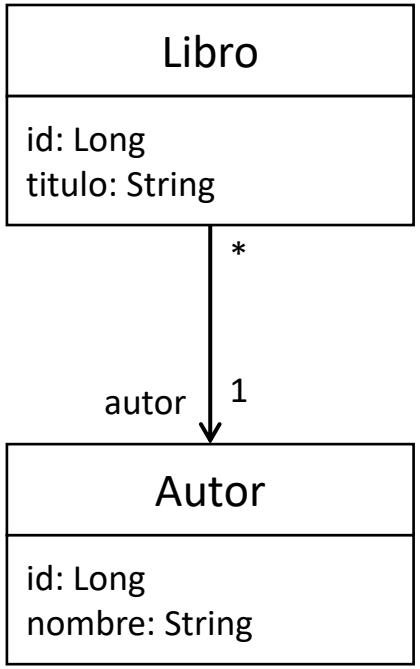
```
@Entity
@DiscriminatorValue("ADMIN")
public class Admin extends Usuario {
. . .
}
```

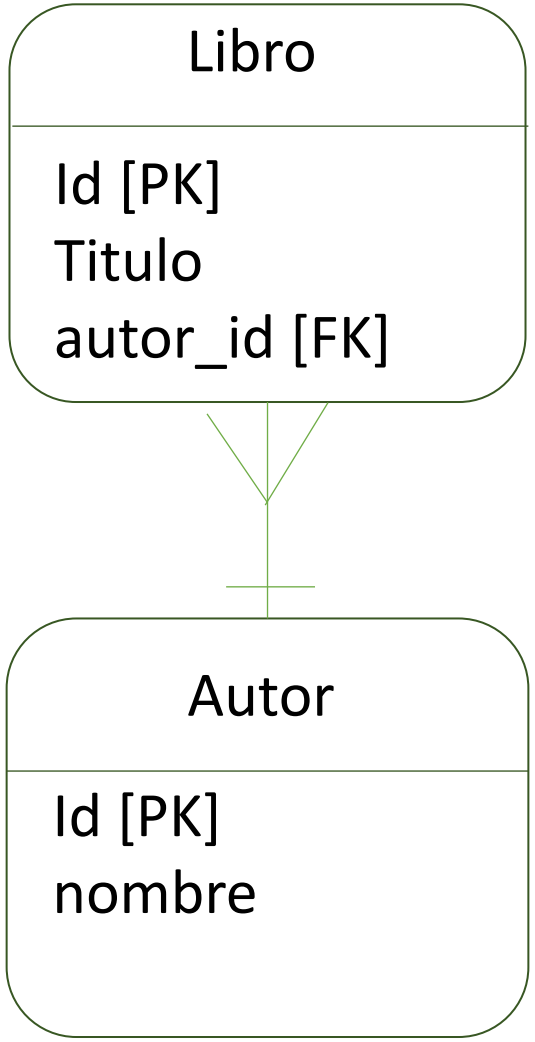
```
@Entity
@DiscriminatorValue("CLIENTE")
public class Cliente extends Usuario {
. . .
}
```

2. Ejemplo: Biblioteca

Relaciones MANY-TO-ONE

- Se anotará en la clase del lado “many”
- Se añadirá una clave ajena en la tabla

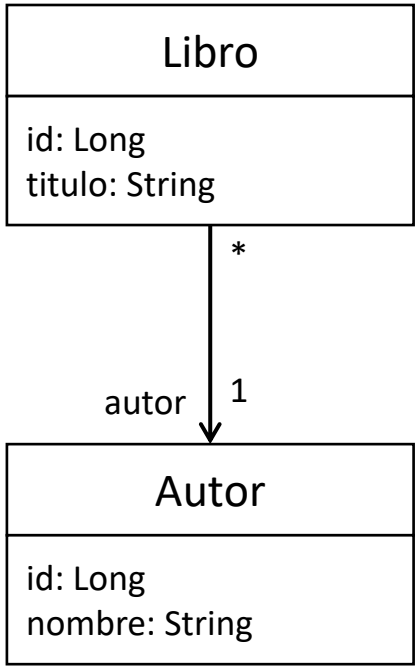




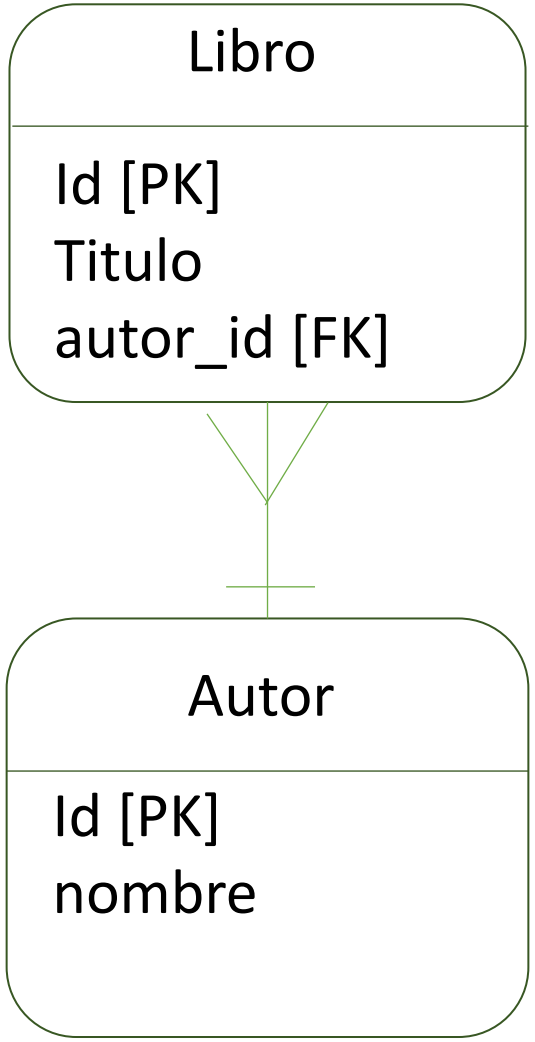
2. Ejemplo: Biblioteca

Relaciones MANY-TO-ONE

- Se anotará en la clase del lado “many”
- Se añadirá una clave ajena en la tabla



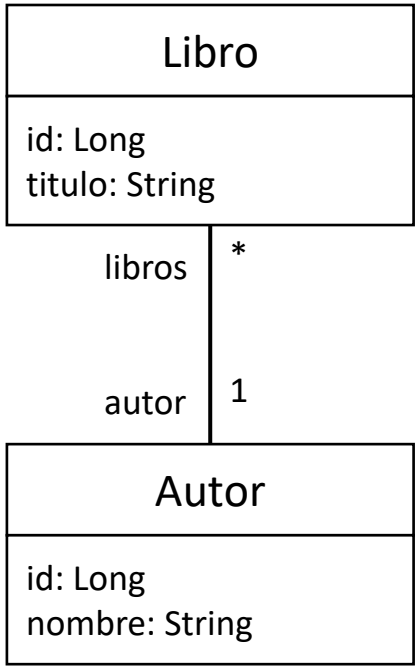

```
@Entity
@Table(name = "libros")
public class Libro {

    . . .

    @ManyToOne
    @JoinColumn(name = "autor_id")
    private Autor autor;

    . . .

}
```

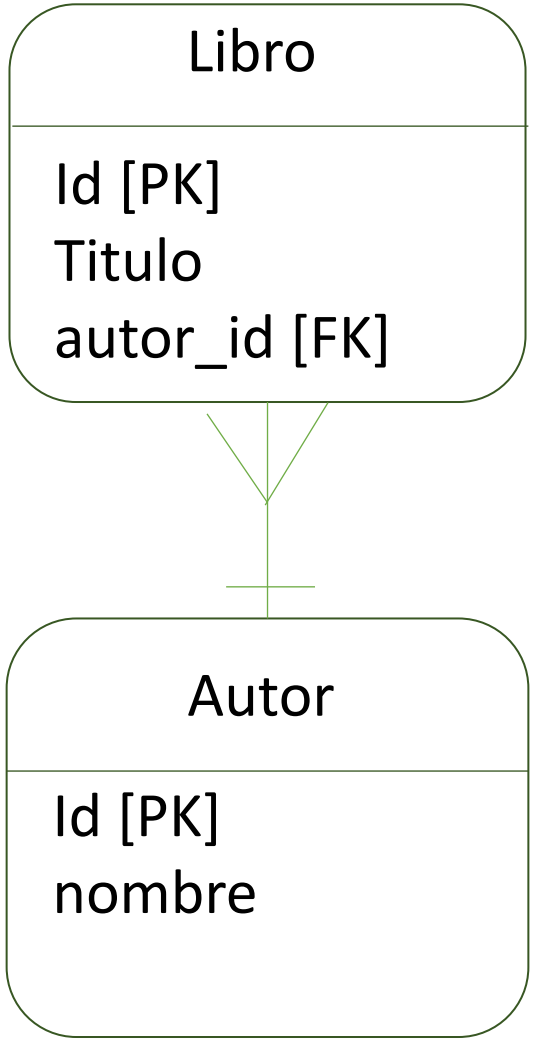


2. Ejemplo: Biblioteca

Relaciones ONE-TO-MANY

- Mismo esquema relacional
- Tan solo anotar (en lado “one”) cuál es el atributo relacionado





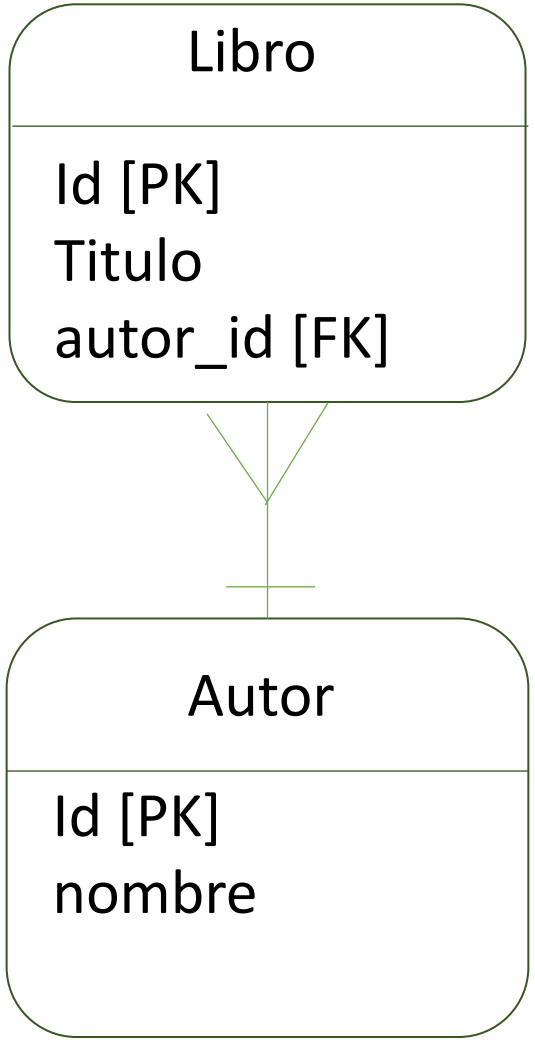
2. Ejemplo: Biblioteca

Relaciones ONE-TO-MANY

- Mismo esquema relacional
- Tan solo anotar (en lado “one”) cuál es el atributo relacionado

```
@Entity
@Table(name = "authors")
public class Autor {
    . . .
    @OneToMany(mappedBy = "autor", cascade = CascadeType.ALL)
    private List<Libro> libros = new ArrayList<Libro>();
    . . .
}
```

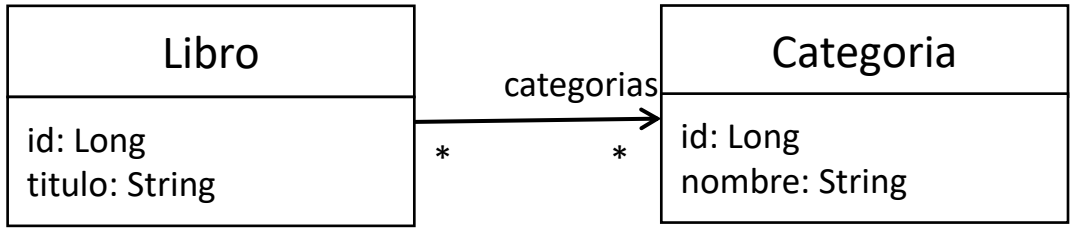
- Si no hay relación @ManyToOne, habría que incluir una anotación @JoinColumn (crearía la misma estructura) o, por defecto, se crearía tabla intermedia.

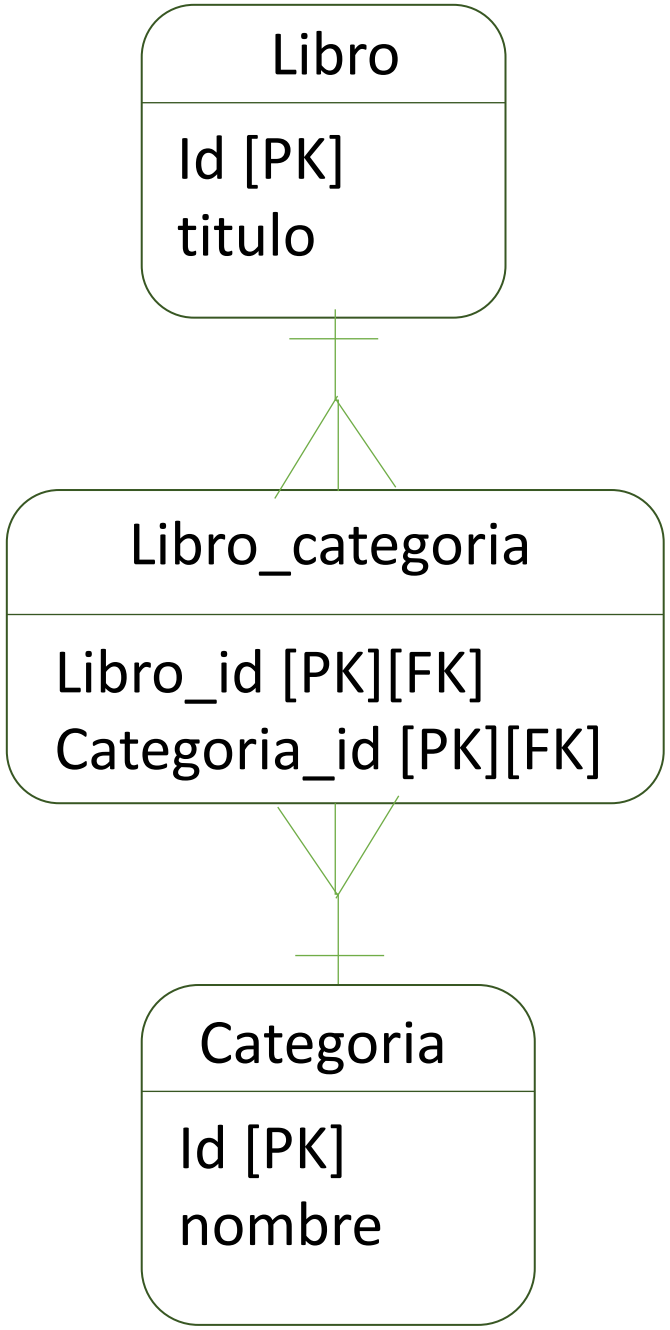


2. Ejemplo: Biblioteca

Relaciones MANY-TO-MANY

- Se crea una tabla intermedia (join table)
- Se anota en la clase que tendrá la colección



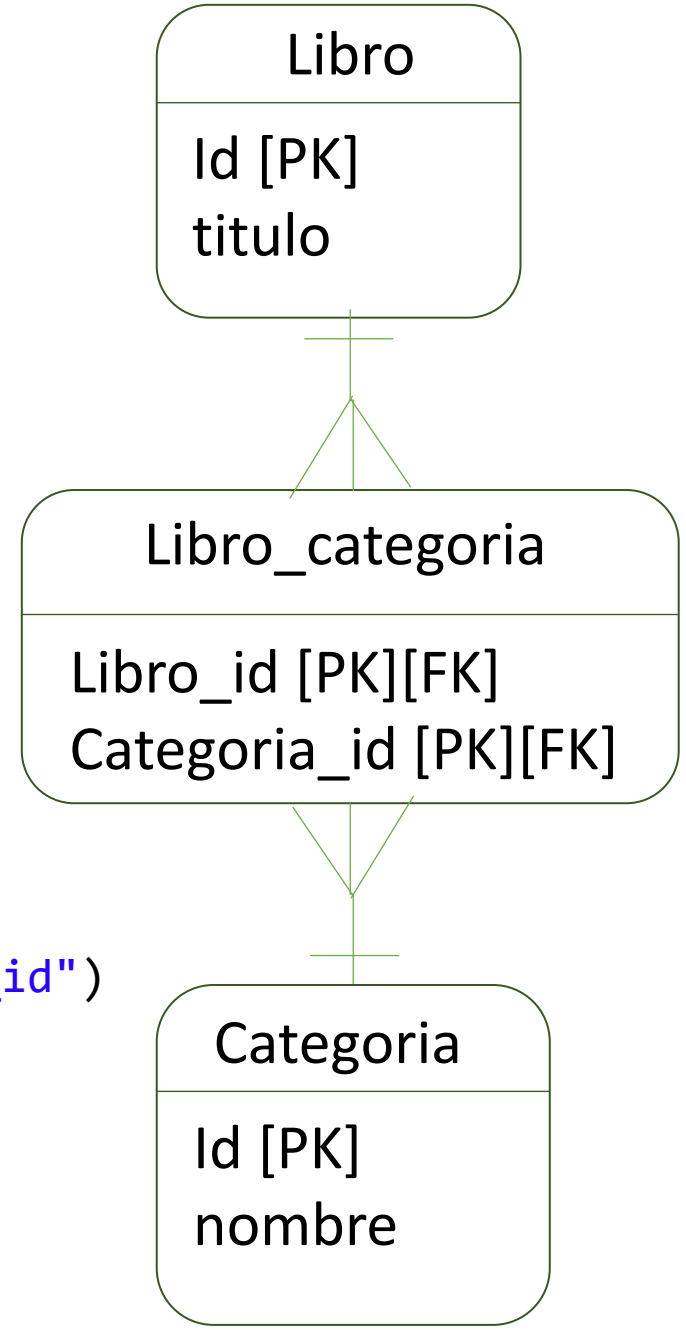


2. Ejemplo: Biblioteca

Relaciones MANY-TO-MANY

- Se crea una tabla intermedia (join table)
- Se anota en la clase que tendrá la colección

```
@Entity
@Table(name = "libros")
public class Libro {
. . .
   @ManyToMany(cascade = CascadeType.ALL)
   @JoinTable(
      name = "libro_categoria",
      joinColumns = @JoinColumn(name = "libro_id"),
      inverseJoinColumns = @JoinColumn(name = "categoria_id")
   )
   private Set<Categoria> categorias;
. . .
}
```

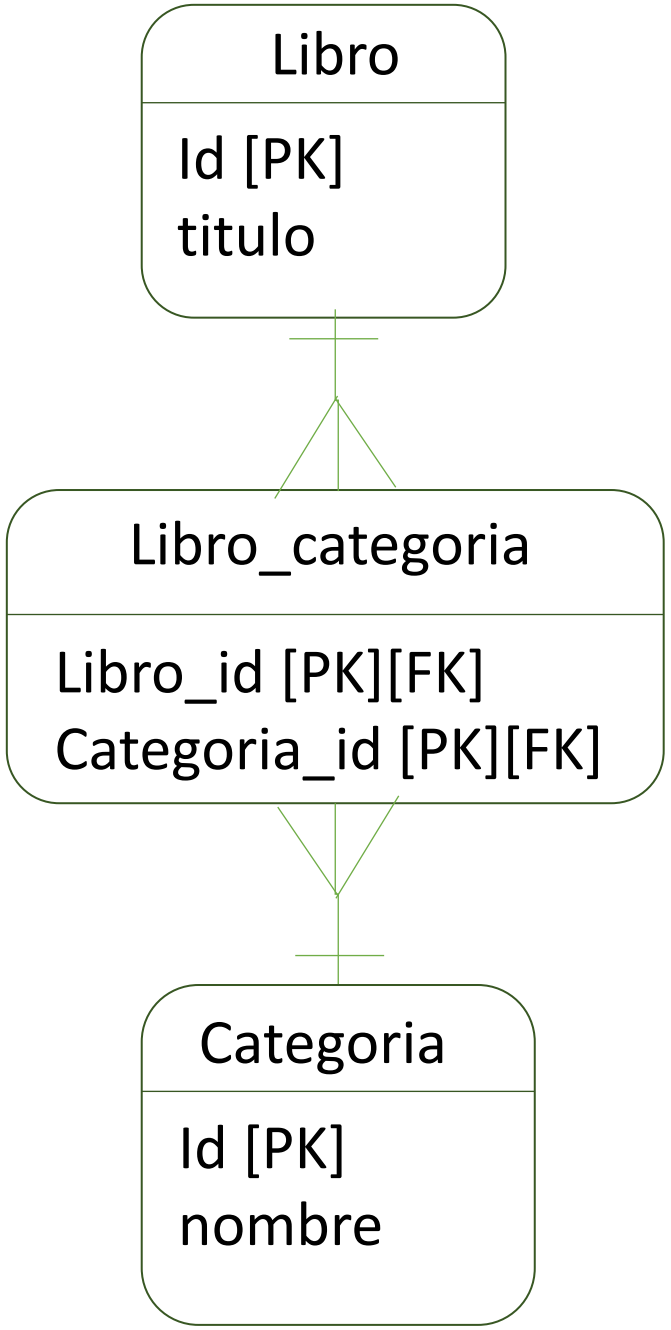


2. Ejemplo: Biblioteca

Relaciones MANY-TO-MANY

- Si fuera bidireccional, en el otro sentido añadiríamos también la anotación, pero con el parámetro @mappedBy

```
@Entity
@Table(name = “categorias")
public class Categoria {
. . .
   @ManyToMany(@mappedBy = “libros”)
   private Set<Libro> libros;
. . .
}
```



2. Ejemplo: Biblioteca – Persistencia de datos

Hasta ahora hemos visto:

- Cómo mapear clases y tablas.
- Cómo representar relaciones.
- Cómo persistir estructuras complejas.

Ahora…

- ¿Quién gestiona los objetos mientras la aplicación está en ejecución?
- ¿Cuándo se sincronizan los cambios con la base de datos?
- ¿Cómo se controla el ciclo de vida de una entidad?

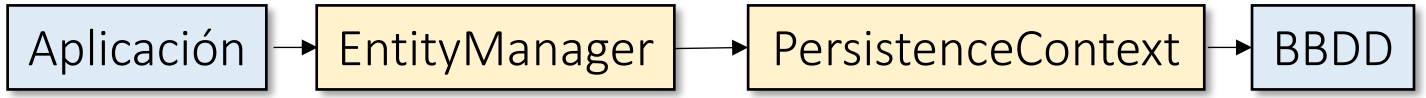
2. Ejemplo: Biblioteca – Entity Manager

El **EntityManager** se utiliza para crear y eliminar instancias de entidades persistentes, para encontrar entidades por su clave principal y para realizar consultas sobre entidades.

Es el **gestor del Contexto de Persistencia**.

Permite:

- Persistir entidades.
- Eliminarlas.
- Buscarlas.
- Sincronizar cambios.
- Controlar transacciones.



https://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html

2. Ejemplo: Biblioteca

Ejemplo simple de persistencia:

```
@Component
public class PersistenciaSimple {
   @PersistenceContext
   private EntityManager em;


   // Las transacciones del EntityManager son gestionadas automáticamente por Spring
   // Es equivalente a em.getTransaction().begin() - código - em.getTransaction().commit();
   @Transactional
   public void ejecutar() {
     // Crear datos de prueba
     Autor author = new Autor("George Orwell");
     em.persist(author);
     . . .
   }
}
```

Lo inyectará SpringBoot cuando sea necesario

3. Configuración

Ejemplo de uso con SQLite y Hibernate

3. Configuración

Dependencias a incluir en nuestro proyecto (ver pom.xml)**:**

- Driver del motor de base de datos a utilizar
- Librería de implementación de JPA a utilizar

3. Configuración

Dependencias a incluir en nuestro proyecto (ver pom.xml):

- **Driver del motor de base de datos a utilizar → SQLite**
- Librería de implementación de JPA a utilizar

```
<!-- SQLite JDBC Driver -->
<dependency>
   <groupId>org.xerial</groupId>
   <artifactId>sqlite-jdbc</artifactId>
   <scope>runtime</scope>
</dependency>
```

Solo necesario en ejecución

3. Configuración

Dependencias a incluir en nuestro proyecto (ver pom.xml):

- Driver del motor de base de datos a utilizar
- **Librería de implementación de JPA a utilizar → Hibernate (por defecto con SpringBoot)**

```
<!-- Spring Boot Data JPA: incluye Hibernate, Spring Data JPA y Spring ORM -->
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Para hacer funcionar SQLite con Hibernate -->
<dependency>
   <groupId>org.hibernate.orm</groupId>
   <artifactId>hibernate-community-dialects</artifactId>
</dependency>
```

3. Configuración

Configuración de la base de datos:

- A través de *application.properties*, como todo en Spring Boot:

URL de configuración de la base de datos. Formato: jdbc:<driver/tipo>:ruta_de_la_bbdd

```
spring.datasource.url=jdbc:sqlite:basedatos.db
spring.datasource.driver-class-name=org.sqlite.JDBC
```

Indica a Spring Boot que usará El driver y dialect de SQLite

```
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Si hay diferencias entre el modelo relacional y las anotaciones de las clases, actualiza el modelo. OJO: cambiar a `validate` en producción

Se mostrarán las consultas SQL que se realicen

4. Consultas con JPQL

Recuperación de datos persistidos con consultas JPQL

4. Consultas con JPQL – Motivación

Java Persistence Query Language (JPQL) es un lenguaje basado en SQL para poder realizar consultas sobre entidades JPA basándose en la estructura de las clases, independientemente de cómo se persistan.

Usaremos:

- Nombres de las clases vs. Nombres de las tablas
- Atributos y colecciones vs. Joins y selects anidados

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultar() {
   // JPQL: nota que usamos el nombre de la clase Java, no el de la tabla
   List<Libro> libros = em.createQuery(
     "SELECT l FROM Libro l WHERE l.autor.name = :nombre", Libro.class)
     .setParameter("nombre", "J.K. Rowling")
     .getResultList();
. . .
}
```

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
     join authors a1_0 on a1_0.id=l1_0.autor_id
where a1_0.name=?
```

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultar() {
   // JPQL: nota que usamos el nombre de la clase Java, no el de la tabla
   List<Libro> libros = em.createQuery(
     "SELECT l FROM Libro l WHERE l.autor.name = :nombre", Libro.class)
     .setParameter("nombre", "J.K. Rowling")
     .getResultList();
. . .
}
```

Necesita todos los atributos de Libro

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
     join authors a1_0 on a1_0.id=l1_0.autor_id
where a1_0.name=?
```

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultar() {
   // JPQL: nota que usamos el nombre de la clase Java, no el de la tabla
   List<Libro> libros = em.createQuery(
     "SELECT l FROM Libro l WHERE l.autor.name = :nombre", Libro.class)
     .setParameter("nombre", "J.K. Rowling")
     .getResultList();
. . .
}
```

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
     join authors a1_0 on a1_0.id=l1_0.autor_id
where a1_0.name=?
```

Hacemos la consulta sobre la clase. Se traduce automáticamente a la tabla donde se está persistiendo

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultar() {
   // JPQL: nota que usamos el nombre de la clase Java, no el de la tabla
   List<Libro> libros = em.createQuery(
     "SELECT l FROM Libro l WHERE l.autor.name = :nombre", Libro.class)
     .setParameter("nombre", "J.K. Rowling")
     .getResultList();
. . .
}
```

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
     join authors a1_0 on a1_0.id=l1_0.autor_id
where a1_0.name=?
```

Condiciones expresadas sobre los atributos de la clase (y sus elementos). Se traducen automáticamente en joins / condiciones SQL

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultar() {
   // JPQL: nota que usamos el nombre de la clase Java, no el de la tabla
   List<Libro> libros = em.createQuery(
     "SELECT l FROM Libro l WHERE l.autor.name = :nombre", Libro.class)
     .setParameter("nombre", "J.K. Rowling")
     .getResultList();
. . .
}
```

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
     join authors a1_0 on a1_0.id=l1_0.autor_id
where a1_0.name=?
```

Usar siempre consultas parametrizadas para evitar errores/ataques de code injection!

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void contarClientes() {
   Long total = em.createQuery(
      "SELECT COUNT(c) FROM Cliente c", Long.class)
      .getSingleResult();
. . .
}
```

SQL ejecutado por Hibernate:

```
select count(c1_0.id)
from usuarios c1_0
where c1_0.tipo_usuario='CLIENTE'
```

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void contarClientes() {
   Long total = em.createQuery(
      "SELECT COUNT(c) FROM Cliente c", Long.class)
      .getSingleResult();
. . .
}
```

SQL ejecutado por Hibernate:

```
select count(c1_0.id)
from usuarios c1_0
where c1_0.tipo_usuario='CLIENTE'
```

Los detalles de la estrategia de persistencia (e.g. herencia en “simple table”) son transparentes

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultarPorCategoria() {
   List<Libro> libros = em.createQuery(
      "SELECT l FROM Libro l JOIN l.categorias c WHERE c.nombre = :cat", Libro.class)
      .setParameter("cat", "Distópico")
      .getResultList();
 . . .
}
```

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
   join libro_categoria c1_0 on l1_0.id=c1_0.libro_id
   join categorias c1_1 on c1_1.id=c1_0.categoria_id
where c1_1.nombre=?
```

4. Consultas con JPQL

Ejemplo en PersistenciaConsultas.java:

```
public void consultarPorCategoria() {
   List<Libro> libros = em.createQuery(
      "SELECT l FROM Libro l JOIN l.categorias c WHERE c.nombre = :cat", Libro.class)
      .setParameter("cat", "Distópico")
      .getResultList();
 . . .
}
```

Usaremos JOIN para buscar en colecciones. Se traducirá a los joins entre tablas necesarios

SQL ejecutado por Hibernate:

```
select l1_0.id,l1_0.autor_id,l1_0.titulo
from libros l1_0
   join libro_categoria c1_0 on l1_0.id=c1_0.libro_id
   join categorias c1_1 on c1_1.id=c1_0.categoria_id
where c1_1.nombre=?
```

5. Repositorios

Implementación de repositorios sencillos con Spring Boot

5. Repositorios

Funciones básicas que siempre suelen estar en un repositorio:

- Guardar un objeto (actualizar si ya existe mismo id)
- Buscar un objecto (por id) / todos los objetos del repositorio
- Borrar un objeto (por id)
- Etc..

5. Repositorios

Funciones básicas que siempre suelen estar en un repositorio:

- Guardar un objeto (actualizar si ya existe mismo id)
- Buscar un objecto (por id) / todos los objetos del repositorio
- Borrar un objeto (por id)
- Etc..

¡ Spring Boot (también) lo hace por ti !

5. Repositorios

Pasos a seguir:

- Crea una interfaz para tu repositorio
- Haz que extienda JpaRepository<Clase_de_objetos_a_persistir, Clase_del_id>
- (Solo si es necesario) Añade métodos de búsqueda adicionales

5. Repositorios

Ejemplo AutorRepository:

```
public interface AutorRepository extends JpaRepository<Autor, Long> {
// Spring genera automáticamente: findAll, findById, save, deleteById, count, ...
}
```

5. Repositorios

Implementación de métodos adicionales:

- Usando nomenclatura de JPA Query Methods (https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html).
  - Multitud de keywords reconocibles. Ej.: findByAgeOrderByLastnameDesc(int age) → -…where x.age = ?1 order by x.lastname desc

- Indicando la consulta JPQL directamente con la anotación @Query

5. Repositorios

Implementación de métodos adicionales:

```
public interface LibroRepository extends JpaRepository<Libro, Long> {

    // Spring genera la consulta a partir del nombre del método
    List<Libro> findByTitulo(String titulo);

    // Consulta JPQL explícita (usando parámetros posicionales, ?1)
    @Query("SELECT l FROM Libro l WHERE l.autor.name = ?1")
    List<Libro> buscarPorNombreAutor(String nombreAutor);

    // Consulta con JOIN a colección (usando parámetros nombrados)
    @Query("SELECT l FROM Libro l JOIN l.categorias c WHERE c.nombre = :categoria")
    List<Libro> buscarPorCategoria(@Param("categoria") String categoria);

    // Consulta con agregación
    @Query("SELECT COUNT(l) FROM Libro l WHERE l.autor.name = :nombre")
    Long contarPorAutor(@Param("nombre") String nombre);
}
```